

POSTER: Security and Cost Trade-offs of Side-Channel Countermeasures for AES Software on RISC-V SoCs

Abolfazl Sajadi
Leiden University
LIACS
Leiden, Netherlands
a.sajadi@liacs.leidenuniv.nl

Todor Stefanov
Leiden University,
LIACS
Leiden, Netherlands
t.p.stefanov@liacs.leidenuniv.nl

Nusa Zidaric
Leiden University
LIACS
Leiden, Netherlands
n.zidaric@liacs.leidenuniv.nl

Nele Mentens
Leiden University
LIACS
Leiden, Netherlands
KU Leuven
ES&S, COSIC
Leuven, Belgium
nele.mentens@kuleuven.be

Abstract

Power side-channel attacks remain a practical concern for cryptographic software deployed on lightweight RISC-V platforms. This poster discusses how several protection strategies influence both leakage resilience and implementation overhead for AES software executed on an FPGA-based RISC-V SoC. Our experiments focus on Tiny-AES running on the Ibex RISC-V core and compare the following protection strategies: software masking, software-generated noise, hardware-generated noise, Secure-Ibex, and CoCo-Ibex. The results show that first-order Correlation Power Analysis succeeds quickly on unprotected SoCs, whereas masking and Secure-Ibex provide the strongest resilience. In contrast, noise-based protection strategies mainly complicate the attack without fully removing exploitable leakage. In addition to security, we compare execution time, code size, hardware usage, power, and energy consumption.

Keywords

power side-channel analysis, RISC-V, FPGA, countermeasures

ACM Reference Format:

Abolfazl Sajadi, Nusa Zidaric, Todor Stefanov, and Nele Mentens. 2026. POSTER: Security and Cost Trade-offs of Side-Channel Countermeasures for AES Software on RISC-V SoCs. In *Proceedings of the 23rd ACM International Conference on Computing Frontiers (CF '26)*, May 19–21, 2026, Catania, Italy. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3801487.3805609>

1 Introduction

Software AES is widely used in embedded RISC-V devices due to its flexibility and lack of dedicated hardware requirements. At the same time, it exposes cryptographic execution to information leakage. Although AES is mathematically secure, its implementation may leak secret information through power consumption.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CF '26, Catania, Italy*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2568-5/26/05

<https://doi.org/10.1145/3801487.3805609>

Correlation Power Analysis (CPA) effectively extracts secrets by exploiting the relationship between power traces and key-dependent intermediate values [2]. Thus, we evaluate countermeasures against CPA within system constraints.

This work compares several representative protection strategies for AES software (Tiny-AES [6] and Mask-AES [3]) on FPGA-based RISC-V SoCs. Rather than proposing new countermeasures, we examine how different software- and hardware-level countermeasures perform under the same CPA attack. The poster summarizes the key ideas and main findings from our complete comparative study [8]. Many side-channel studies focus on one countermeasure at a time, making it difficult to judge which solution is more attractive for a real system implementation. In practice, designers must choose among imperfect solutions with different cost–security trade-offs.

2 Evaluated Strategies

We evaluate and compare the following system implementations:

- **Tiny-AES on Ibex**¹: unprotected baseline implementation;
- **Mask-AES**: boolean-masked AES software;
- **SW-NG**: software noise generated by duplicating the S-box computation with a different key;
- **HW-NG**: hardware noise generated through SRL-based [5] activity in the FPGA;
- **Secure-Ibex**: Ibex with data-independent timing and random dummy instruction insertion;
- **CoCo-Ibex**: a processor variant supporting masked software by reducing leakage-related micro-architectural effects [4]

The evaluated implementations include masking (randomizing sensitive intermediate values to break statistical correlations) and hiding (distorting leakage via switching activity or varying execution timing). Since these strategies affect the leakage differently, their cost and effectiveness can differ significantly even when they protect the same algorithm. This is particularly relevant for lightweight systems, where even a modest increase in area, execution time, or energy consumption can determine whether a strategy is useful in

¹<https://github.com/lowRISC/ibex>

Table 1: Classification of evaluated strategies

Category	Hardware	Software
Algorithm-independent	Secure-Ibex, HW-NG	SW-NG
Algorithm-aware	CoCo-Ibex	Mask-AES

practice. The aforementioned strategies span several useful classes, as shown in Table 1: algorithm-aware/independent, software-level, and hardware-level protection. This classification is the basis of our comparison.

3 Experimental Setup

The experimental setup consists of two parts: a *Control SoC* and a *Target SoC*. The Control SoC manages memory loading and data communication, and controls the overall experiment. The Target SoC executes the AES software on one of the target RISC-V cores. Depending on the experiment, the target RISC-V core is synthesized as Ibex, Secure-Ibex, or CoCo-Ibex. The Target and Control SoCs are implemented on a CW305 board containing an Artix-7 FPGA and clocked at 50 MHz. Power traces are captured with ChipWhisperer-Husky [7] at 200 MHz when the first-round AES S-box computation is executed, which is a standard practice for first-order CPA on software AES.

Fairness is ensured by analyzing all implementations on the same hardware, with identical trace acquisition and CPA workflow. This minimizes setup-dependent variation and attributes differences primarily to the countermeasures.

Scope of the experiments. Our experiments evaluate both protection effectiveness and implementation cost for each strategy. Regarding protection effectiveness, we check whether first-order CPA can recover the correct AES key. Regarding implementation cost, we evaluate code size, execution time, FPGA resource usage, power, and energy consumption. This joint view is essential because a protection strategy that reduces information leakage may still be impractical if its implementation cost is too high for a constrained system. In addition, the reported results should be interpreted within the considered attack model, namely CPA targeting the first S-box. The comparison is highlighting the evaluated protection strategies under a realistic baseline attack, instead of reasoning about higher-order, profiling, or combined attack strategies.

4 Experimental Results

The experimental results are summarized in Table 2. The baseline system implementation, **Tiny-AES on Ibex**, is clearly vulnerable. The correct encryption key byte can be recovered for all 16 subkeys by capturing and analyzing only 250 power traces.

Mask-AES provides the strongest software-level protection in our experiments. Even with 21,000 traces, first-order CPA does not reveal a distinguisher for any subkey. This security gain comes at a cost: the code size grows by 2.03× and execution time by 1.47×.

SW-NG and **HW-NG** both make the attack harder, but do not fully suppress the information leakage. In SW-NG, the duplicated S-box introduces a competing correlation source, which creates ambiguity rather than full protection. In HW-NG, the added hardware noise lowers the signal-to-noise ratio and increases the number of

Table 2: Protection effectiveness and implementation cost

Configuration	CPA result	Main overhead
Ibex + Tiny-AES	≈250 traces per subkey	Baseline
Ibex + Mask-AES	No recovery in 21K traces	2.03× code, 1.47× time
Ibex + SW-NG	Ambiguous peaks; recoverable	1.34× controller code
Ibex + HW-NG	≥400 traces in some cases	3.98× LUTs
Secure-Ibex-min	No recovery in 18K traces	1.41× time
Secure-Ibex-max	No recovery in 18K traces	5.13× time
CoCo-Ibex + Tiny-AES	≈250 traces per subkey	1.4× LUTs, 1.52× FFs

traces needed for key recovery. Still, the correct-key trend remains observable as the captured trace set grows.

Secure-Ibex shows the strongest overall protection in our setup. By combining data-independent timing with randomized dummy instruction insertion, it reduces the temporal regularity that CPA typically exploits. For both tested instruction-insertion settings, no clear key distinguisher appears within 18,000 traces, but this comes at the cost of a substantial increase in execution time (up to 5.13×).

CoCo-Ibex does not improve security over the unprotected baseline; key recovery is still observed. This is because CoCo-Ibex is designed to support masked software rather than securing unmasked AES directly.

Implementation cost. Compared protection strategies stress the system in very different ways. Masking mainly increases the software cost, Secure-Ibex mainly increases execution time and energy consumption, and HW-NG mainly increases FPGA resource usage in terms of LUTs. These different overhead profiles are an important consideration during system integration.

5 Conclusions

This poster presents a comparison of several protection strategies against CPA-based attacks on AES software running on an FPGA-based RISC-V SoC. For this experimental setup, the most effective strategies are masking of the AES software and Secure-Ibex, each with a distinct overhead profile. The experimental results show that no single strategy is universally best. Masking provides strong software-level resilience to first-order attacks but increases code size and execution time. Secure-Ibex provides strong protection at the cost of execution time and energy consumption. Noise-based strategies are somewhat lighter, but they mostly delay the attack instead of preventing it.

Acknowledgment This work was funded by the Dutch Research Council (NWO) through the PROACT project [1] (NWA.1215.18.014).

References

- [1] Asmita Adhikary et al. 2024. PROACT: Physical Attack Resistance of Cryptographic Algorithms and Circuits with Reduced Time to Market. In *ARC'24*. 255–266.
- [2] Eric Brier, Christophe Clavier, and Francis Olivier. 2004. Correlation Power Analysis with a Leakage Model. In *CHES'04*.
- [3] CENSUS. 2020. masked-aes-c. <https://github.com/CENSUS/masked-aes-c>.
- [4] Barbara Gigerl, Vedad Hadzic, Robert Primas, Stefan Mangard, and Roderick Bloem. 2021. CoCo: Co-Design and Co-Verification of Masked Software Implementations on CPUs. In *USENIX Security '21*. 1469–1468.
- [5] Tim Gueneysu and Amir Moradi. 2011. Generic Side-Channel Countermeasures for Reconfigurable Devices. In *CHES'11*. 33–48.
- [6] Kokke. 2019. tiny-AES-c. <https://github.com/kokke/tiny-AES-c>.
- [7] Colin O'Flynn and Zhizhang Chen. 2014. ChipWhisperer: An Open-Source Platform for Hardware Embedded Security Research. In *COSADE'14*. 243–260.
- [8] Abolfazl Sajadi, Nusa Zidaric, Todor Stefanov, and Nele Mentens. 2024. A Systematic Comparison of Side-Channel Countermeasures for RISC-V-Based SoCs. In *NorCAS'24*. 1–7. doi:10.1109/NorCAS4408.2024.10752477